

Claude Code Cheatsheet

L'essentiel au quotidien pour une productivité maximale

v3.43.0 | septembre 2026

Installation

```
npm install -g @anthropic-ai/claude-code
# Vérifier l'installation which claude && claude
--version
```

Contrôle santé : `claude doctor && claude mcp list`

Commandes essentielles

Commande	Action
<code>/help</code>	Aide contextuelle
<code>/powerup</code>	Leçons animées interactives sur les fonctionnalités de Claude Code
<code>/clear</code>	Réinitialiser la conversation
<code>/compact</code>	Libérer du contexte
<code>/status</code>	État de la session + usage du contexte
<code>/context</code>	Détail de la consommation de tokens
<code>/plan</code>	Entrer en Plan Mode (aucune modification)
Approuver le plan	Sortir du Plan Mode (<code>Shift+Tab</code> pour sortir sans valider)
<code>/model</code>	Choisir le modèle ; Entrée enregistrée, s limite à la session
<code>/insights</code>	Analytique d'usage + rapport d'optimisation
<code>/simplify</code>	Revue de nettoyage uniquement sur le code modifié : réutilisation, simplification, efficacité
<code>/batch</code>	Refactors à grande échelle via 5 à 30

Commande	Action
	agents en worktrees parallèles
<code>/teleport</code>	Téléporter une session depuis le web
<code>/tasks</code>	Suivre les tâches en arrière-plan
<code>/remote-env</code>	Configurer un environnement cloud
<code>/rc</code>	Démarrer une session de contrôle à distance (Pro/Max/Team/Enterprise)
<code>/mobile</code>	Récupérer les liens de téléchargement de l'app mobile Claude
<code>/fast</code>	Basculer le mode rapide (Opus 5.5 : \$8/\$40 par MTok)
<code>/voice</code>	Activer l'entrée vocale (maintenir Espace pour parler)
<code>/recap</code>	Résumé de contexte de session au retour d'une pause
<code>/effort [niveau]</code>	Profondeur de réflexion : low/medium/high/xhigh/max
<code>/loop [intervalle] [prompt]</code>	Répète un prompt en boucle (rythme autonome sans intervalle)
<code>/goal [condition]</code>	Mode multi-tours autonome jusqu'à ce que la condition soit remplie
<code>/rename [nom]</code>	Nommer ou renommer la session en cours
<code>/debug</code>	Débogage systématique
<code>/exit</code>	Quitter (ou Ctrl+D)

Raccourcis clavier

Raccourci	Action
Shift+Tab	Alternier les modes de permission
Esc × 2	Rewind (annuler)
Ctrl+C	Interrompre
Ctrl+R	Rechercher dans l'historique des commandes
Ctrl+L	Effacer l'écran (garde le contexte)
Ctrl+B	Tâches en arrière-plan
Ctrl+X Ctrl+K	Tuer tous les agents en arrière-plan
Alt+T	Basculer la réflexion si le modèle le permet
Espace (maintenu)	Entrée vocale (nécessite /voice)
Tab	Autocomplétion
Shift+Enter	Nouvelle ligne
Ctrl+D	Quitter

Raccourcis IDE : VS Code `Alt+K` | JetBrains `Cmd+Option+K`

Références de fichiers

@chemin/vers/fichier.ts	→ Référencer un fichier
@nom-agent	→ Appeler un agent
!commande-shell	→ Exécuter une commande shell

Fonctionnalités méconnues (mais officielles !)

Fonctionnalité	Ce que ça fait
Tasks API	Listes de tâches persistantes avec dépendances
Background Agents	Des sous-agents travaillent pendant que vous codez
Agent Teams	Coordination multi-agents (TeamCreate/SendMessage)
Auto-Memories	Capture automatique du contexte entre sessions
Session Forking	Rewind + création d'une timeline parallèle
LSP Tool	Intelligence de code (go-to-def, refs), ~50ms contre 45s avec grep

Fonctionnalité	Ce que ça fait
Mode vocal	Entrée vocale native, transcription gratuite
Remote Control	Contrôler une session locale depuis téléphone/navigateur (Pro/Max/Team/Enterprise)
Tâches planifiées (Cloud/Desktop)	Planification machine éteinte via <code>/schedule</code> ou l'app Desktop
Skill Evals	Tests A/B et calibrage des déclencheurs pour les skills personnalisées
Output Styles	<code>/output-style</code> : Default, Proactive, Concise, Explanatory, Learning
Rules Templates	Templates <code>.claude/rules/</code> prêts à l'emploi (arch/qualité/tests/perf)

Astuce : ce ne sont pas des « secrets », tout est dans le [CHANGELOG](#). Lisez-le !

Modes de permission

Mode	Édition	Exécution
Manual (default)	Demande	Demande
acceptEdits Auto		Demande
Plan Mode	Aucune	Outils en lecture seule
auto	Le classifieur décide	Le classifieur décide
dontAsk	Uniquement si dans les règles autorisées	Uniquement si dans les règles autorisées
bypassPermissions	Auto	Auto (CI/CD uniquement)

Shift+Tab pour changer de mode

Mémoire & configuration (2 niveaux)

Niveau	Chemin	Portée	Git
Projet	<code>.claude/</code>	Équipe	Oui
Personnel	<code>~/.claude/</code>	Vous (tous projets)	Non

Priorité : le projet prime sur le personnel

Structure du dossier `.claude/`

<code>.claude/</code>	├─ CLAUDE.md	# Mémoire locale (gitignored)
	├─ settings.json	# Hooks (versionné)

└─ settings.local.json # Permissions (non versionné)

└─ agents/ # Agents personnalisés

└─ hooks/ # Scripts d'événements

└─ rules/ # Règles chargées automatiquement

└─ architecture-review.md

└─ code-quality-review.md

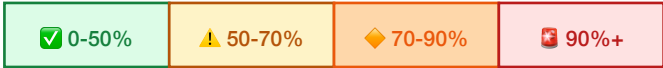
└─ test-review.md

└─ performance-review.md

└─ skills/ # Slash commands + modules de connaissance (unifiés)

Gestion du contexte (CRITIQUE)

Statusline : Model: Sonnet | Ctx: 89.5k | Ctx(u): 56.0%



Surveillez Ctx(u): → >70% = /compact , >85% = /clear

Signe	Action
Réponses courtes	/compact
Oublis fréquents	/clear
Contexte >70%	/compact
Tâche terminée	/clear

Commandes de récupération du contexte

Commande	Usage
/compact	Résumer et libérer du contexte
/clear	Repartir de zéro
/rewind	Annuler les changements récents
claude -c	Reprendre la dernière session
claude -r <id>	Reprendre une session spécifique

Plan Mode & réflexion

Fonctionnalité	Activation	Usage
Plan Mode	Shift+Tab × 2 ou /plan	Explorer sans modifier
OpusPlan	/model opusplan	Opus pour planifier, Sonnet pour exécuter

Thinking et effort

Opus 5.5, Sonnet 5 et Fable 5.1 utilisent le thinking adaptatif. Haiku 4.5 dispose du thinking étendu mais pas du paramètre effort.

Les modèles adaptatifs actuels acceptent low , medium , high , xhigh et max . Opus 4.6 et Sonnet 4.6 acceptent max , mais pas xhigh .

/effort enregistre les niveaux jusqu'à xhigh par modèle. s limite le choix à la session. max reste propre à la session ; /effort auto efface le choix enregistré pour le modèle actif.

Option+T / Alt+T bascule le thinking si le modèle le permet. Ce raccourci n'a pas d'effet sur Opus 5.5 ou Fable, dont le thinking reste actif. Réduire plutôt l'effort pour une tâche bornée.

ultrathink ajoute une consigne de réflexion sans changer l'effort API. think hard est du texte ordinaire.

Workflow typique

1. Démarrer la session → claude

2. Vérifier le contexte → /status

3. Plan Mode → Shift+Tab × 2 (tâches complexes)

4. Décrire la tâche → prompt clair et précis

5. Vérifier les diffs → toujours lire avant d'accepter !

6. Accepter/Rejeter → y/n

7. Vérifier → lancer les tests

8. Commit → une fois la tâche terminée

9. /compact → quand le contexte dépasse 70%

Formule de prompt rapide

QUOI : [Livrable concret] OÙ : [Chemins de fichiers]

COMMENT : [Contraintes, approche] VÉRIFIER : [Critères de succès]

Exemple :

Ajouter une validation d'entrée au formulaire de connexion.

OÙ : src/components/LoginForm.tsx

COMMENT : utiliser un schéma Zod, afficher les erreurs inline

VÉRIFIER : email vide affiche une erreur, format invalide affiche une erreur

Serveurs MCP

Serveur	Usage
Serena	Indexation + mémoire de session + recherche de symboles
grepai	Recherche sémantique + analyse du graphe d'appels
Context7	Documentation des librairies
Sequential	Raisonnement structuré
Playwright	Automatisation navigateur
Postgres	Requêtes base de données

Serveur	Usage
doobidoo	Mémoire sémantique + Knowledge Graph

Mémoire Serena : `write_memory()` / `read_memory()` / `list_memories()`

Vérifier l'état : `/mcp`

Outils de recherche, aide-mémoire

Décision rapide : texte exact → `rg` | nom exact → `rg` /
Serena | concept → `grepai` | structure → `ast-grep`

Tâche	Outil
Trouver du texte exact	<code>rg "TODO"</code>
Trouver du code lié à l'auth	<code>grepai search "authentication"</code>
Qui appelle une fonction	<code>grepai trace callers "login"</code>
Structure d'un fichier	<code>serena get_symbols_overview</code>

Aide-mémoire des flags CLI

Flag	Usage
<code>-p "requête"</code>	Mode non-interactif (CI/CD)
<code>-c</code> / <code>--continue</code>	Continuer la dernière session
<code>-r</code> / <code>--resume <id></code>	Reprendre une session spécifique
<code>--teleport</code>	Téléporter une session depuis le web
<code>--model sonnet</code>	Changer de modèle
<code>--add-dir ../lib</code>	Autoriser l'accès hors du répertoire courant
<code>--permission-mode plan</code>	Plan mode
<code>--worktree</code> / <code>-w</code>	Lancer dans un git worktree isolé
<code>--max-budget-usd 5.00</code>	Plafond de dépense API (mode print)
<code>--dangerously-skip-permissions</code>	Auto-accepter (à manier avec précaution)
<code>--debug='mcp'</code>	Déboguer les serveurs MCP
<code>--allowedTools "Edit,Read"</code>	Liste blanche d'outils
<code>--debug</code>	Sortie de débogage

Anti-patterns

❌ Don't	✅ Do
- Prompts vagues - Accepter sans lire - Ignorer les avertissements - Sauter les permissions - Contraintes négatives uniquement	- Préciser fichier + ligne avec @références - Lire chaque diff - Utiliser <code>/compact</code> à 70% - Jamais en production - Fournir des alternatives

Optimisation des coûts

Modèles actuels

Tarifs API standard vérifiés le 24 septembre 2026.
Dollars par million de tokens en entrée/sortie, hors cache.

Modèle	Entrée / sortie	Contexte	Effort par défaut
Haiku 4.5	\$1 / \$5	200K	Non pris en charge
Sonnet 5	\$2 / \$10	1M	high
Opus 5.5	\$4 / \$20	1M	medium
Fable 5.1	\$10 / \$50	1M	high

Le tarif publié de Sonnet reste \$2/\$10 après la promotion de lancement. Opus 5.5 est le défaut du service direct, sauf règle du compte. Le modèle seul ne détermine pas le coût d'une tâche.

Modèle	Usage	Coût
Haiku	Corrections simples, revues	\$
Sonnet	La majorité du développement	\$\$
Opus	Architecture, bugs complexes	\$\$\$
OpusPlan	Plan (Opus) + Exécution (Sonnet)	\$\$

Arbre de décision rapide

Carte rapide des quatre couches

Couche	Question de choix	Référence
Modèle	Quel raisonnement et quel coût conviennent?	Glossaire

Couche	Question de choix	Référence
Runtime harness	Qui exécute la boucle outils-observation-décision?	Agent Harness Map
Repository harness	Quelles instructions, vérifications et états encadrent le dépôt?	Agent Harness Engineering
Orchestrateur	Faut-il coordonner plusieurs runtimes ou sessions?	Agent Tools

Évaluer le couple modèle-harness exact pour une tâche de code. Ajouter un orchestrateur lorsque la coordination est le besoin identifié. Un optimiseur de harness reste hors des quatre couches opérationnelles et modifie des harnesses candidats sous un protocole d'évaluation séparé.

Boucle et graphe ne sont pas des substituts de plus. La boucle nomme le feedback répété et la règle d'arrêt. Le graphe nomme les nœuds, branches et checkpoints exécutables dans la structure choisie. Pour les mécanismes observés, `confirmed` = mécanisme visible, `claimed` = déclaration source, `unknown` = point non établi. Voir aussi [Loop & Graph Engineering](#).

Tâche simple	→ demander directement à Claude
Tâche complexe	→ passer par Tasks API pour planifier
Changement risqué	→ Plan Mode d'abord
Tâche répétitive	→ créer un agent ou une commande
Contexte plein	→ <code>/compact</code> ou <code>/clear</code>
Besoin de doc	→ utiliser Context7 MCP
Analyse profonde	→ utiliser Opus (réflexion activée par défaut)

The Golden Rules

- 1 Toujours relire les diffs avant d'accepter
- 2 Utiliser `/compact` avant que le contexte devienne critique (>70%)
- 3 Être précis dans les demandes (QUOI, OÙ, COMMENT, VÉRIFIER)
- 4 Plan Mode d'abord pour les tâches complexes ou risquées
- 5 Créer un CLAUDE.md pour chaque projet
- 6 Commit fréquemment après chaque tâche terminée
- 7 Savoir ce qui est envoyé : prompts, fichiers, résultats MCP vers Anthropic

Résolution rapide des problèmes courants

Problème	Solution
« Command not found »	Vérifier le PATH, réinstaller npm global
Contexte trop élevé (>70%)	<code>/compact</code> immédiatement
Réponses lentes	<code>/compact</code> ou <code>/clear</code>
MCP ne fonctionne pas	<code>claude mcp list</code> , vérifier la config
Permission refusée	Vérifier <code>settings.local.json</code>
Hook bloquant	Vérifier le code de sortie du hook, revoir la logique

Contrôle santé : `which claude && claude doctor && claude mcp list`

Auteur : Florian BRUNIAUX | [Méthode Aristote](#) | Écrit avec Claude

Version 3.43.0 | Septembre 2026